CS402 – High-Performance Computing

OpenMP

Gihan Mudalige
Professor
Department of Computer Science
University of Warwick
g.mudalige@warwick.ac.uk

OpenMP – BSP on Multi-cores

- OpenMP realizes the BSP / fork-join model for multi-core CPUs
  - Done through the introduction of **compiler directives** / **`pragmas`**

- A `pragma` is a **comment** in a special format
  - Passes hints to the compiler how to handle certain code parts
  - Allows to add information to the code that we cannot supplement with built-in capabilities of the language
  - Compiler ignores the `pragma` if its not known
  - Program needs to be compiled in a certain way (e.g. flags) to action the `pragma`

$$\underbrace{\texttt{\#pragma}}_{\text{C's pragma keyword.}}\ \underbrace{\texttt{omp}}_{\text{OpenMP identifier.}}\ \underbrace{\texttt{directive}}_{\text{What do to.}}\ \underbrace{\texttt{clause}}_{\text{How do to it (optional).}}$$

- OpenMP defines **prescriptive annotations**
  - enforces a specific transformation (e.g. paralleize, vectorise)
  - As opposed to **descriptive annotation –** hints to create efficient code which the compiler can ignore

OpenMP Philosophy
Stick as close to a serial code as possible and add all parallelization through `pragmas` only

- Consider following code from a simple N-Body problem

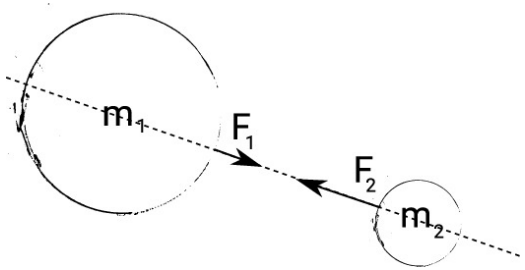
```
// only the declaration here
void calcForce(
  double pAx, double pAy, double pAz,   // position body A [in]
  double pBx, double pBy, double pBz,   // position body B [in]
  double& Fx, double& Fy, double& Fz    // force [out]
);

// time stepping loop
void simulate(...) {
  double t = 0.0;

  while (t < T) {
    for (int n = 0; n < N; n++) { // for every body in system
      // reset forces
      force[n][0] = 0.0;
      force[n][1] = 0.0;
      force[n][2] = 0.0;

      for (int m2 = 0; m2 < N; m2++) { // compute interactions
        if (n != m2) {
          double Gx, Gy, Gz;
          calcForce( p[n][0],  p[n][1],  p[n][2],
                     p[m2][0], p[m2][1], p[m2][2],
                     Gx, Gy, Gz );

          force[n][0] += Gx;
          force[n][1] += Gy;
          force[n][2] += Gz;
        }
      }
    }
    // update particles (don't mix)
    for (int n = 0; n < N; n++) { // for every body in system
      p[n][0] += dt * v[n][0];
      p[n][1] += dt * v[n][1];
      p[n][2] += dt * v[n][2];
      v[n][0] += dt * force[n][0] / m[n];
      v[n][1] += dt * force[n][1] / m[n];
      v[n][2] += dt * force[n][2] / m[n];
    }
    t += dt;
  }
}
```



❑ Add OMP `pragma`

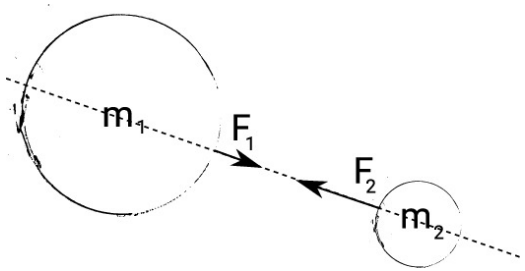
```
// only the declaration here
void calcForce(
  double pAx, double pAy, double pAz,   // position body A [in]
  double pBx, double pBy, double pBz,   // position body B [in]
  double& Fx, double& Fy, double& Fz    // force [out]
);

// time stepping loop
void simulate(...) {
  double t = 0.0;

  while (t < T) {
    #pragma omp parallel for
    for (int n = 0; n < N; n++) { // for every body in system
      // reset forces
      force[n][0] = 0.0;
      force[n][1] = 0.0;
      force[n][2] = 0.0;

      for (int m2 = 0; m2 < N; m2++) { // compute interactions
        if (n != m2) {
          double Gx, Gy, Gz;
          calcForce( p[n][0],  p[n][1],  p[n][2],
                     p[m2][0], p[m2][1], p[m2][2],
                     Gx, Gy, Gz );

          force[n][0] += Gx;
          force[n][1] += Gy;
          force[n][2] += Gz;
        }
      }
    }
    // update particles (don't mix)
    #pragma omp parallel for
    for (int n = 0; n < N; n++) { // for every body in system
      p[n][0] += dt * v[n][0];
      p[n][1] += dt * v[n][1];
      p[n][2] += dt * v[n][2];
      v[n][0] += dt * force[n][0] / m[n];
      v[n][1] += dt * force[n][1] / m[n];
      v[n][2] += dt * force[n][2] / m[n];
    }
    t += dt;
  }
}
```



Compiling and Running

- For Fortran, C and C++, **OpenMP is a standard** that almost all compilers implement
  - Version 1.0 for Fortran came in 1997, C/C++ the following year
  - Version 3.1 (2009) – most widely used / implemented by compilers

- To *action* the `pragma` compile code with flags :

	GCC	Clang	Intel (OneAPI)	NVIDIA (nvcc)	AMD (aocc)	IBM XL
Flag(s)	`-fopenmp`	`-fopenmp -lomp`	`-qopenmp`	`-mp`	`-fopenmp`	`-qsmp=omp`

```
# GCC
gcc -fopenmp main.c -o main

# Clang
clang -fopenmp main.c -lomp -o main

# Intel oneAPI
icx -qopenmp main.c -o main

# NVIDIA
nvc++ -mp main.cpp -o main
```

```
# set number of threads available
export OMP_NUM_THREADS=3

# run
./main
```

OpenMP – BSP on Multi-cores

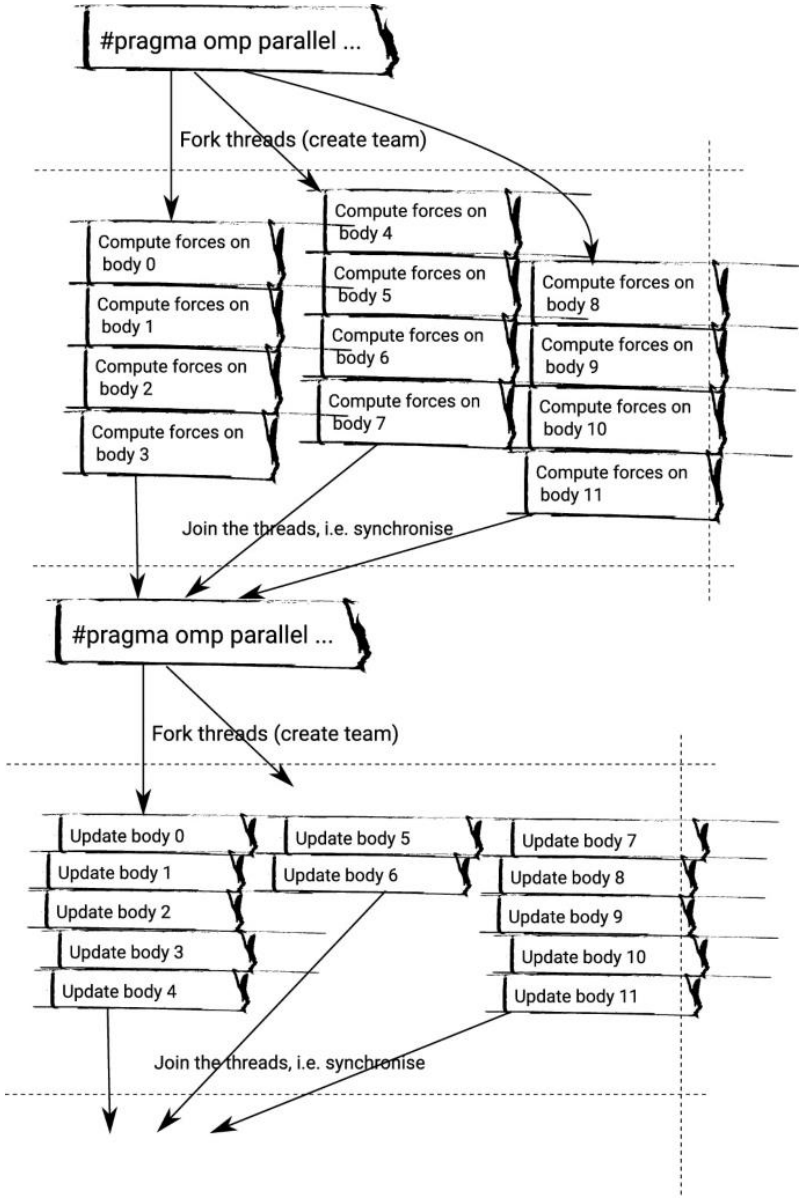
- Straightforward BSP parallelisation
  - **Two super-steps** : the two loops over each body

```
13    #pragma omp parallel for
14    for (int n = 0; n < N; n++) { // for every body in system

34    #pragma omp parallel for
35    for (int n = 0; n < N; n++) { // for every body in system
```

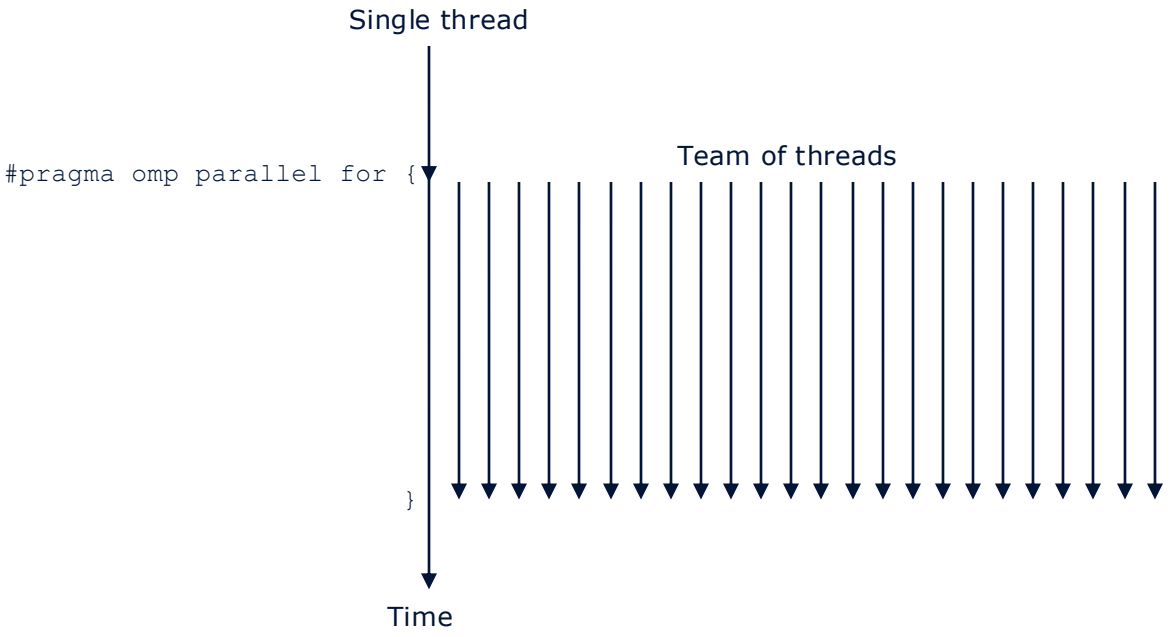
- Solve for **12 bodies** on **3 CPU cores**
  - For each super-step – **spawn** three OpenMP *threads*
  - OpenMP **automatically splits** up the for-range into three chunks.
  - A thread completing its chunk waits for others to finish (*join*) – this is **implicit** at the end of the for loop

- An OpenMP thread is a runtime-managed **thread of execution** within a shared-memory process
  - created at the start of a parallel region
  - shares the program's address space while maintaining a private execution state



OpenMP – BSP on Multi-cores

- Entering a `pragma` block
  - OpenMP initiates `OMP_NUM_THREADS` number of threads (a *team* of threads)
  - `OMP_NUM_THREADS` set at runtime – not hard-coded, thus **platform independent**



- Modern OpenMP well optimized for administering the *team of threads* created
  - Historically threads were created and destroyed at the end of the `pragma` region
  - Causes **overhead**
  - Modern OpenMP implementations lets thread **idle and reuse** – wake up threads at a parallel region

Combined Constructs

- **`#pragma omp parallel for`** is a combination of :
  - `#pragma omp parallel`
  - `#pragma omp for`
  - Ensures that **loop range is properly split** up and **loop iterations are distributed** among threads

```
#pragma omp parallel {

  #pragma omp for
  for (int i=0; i<N; i++) { // for every body in system
    [...]
  } // update particles (don't mix)

  #pragma omp for
  for (int i=0; i<N; i++) { // for every body in system
    [...]
  }
}
```

- If you forget the `#pragma omp for` – the whole for loop will be executed by each of the threads !
  - i.e. run the loop OMP_NUM_THREADS times in parallel

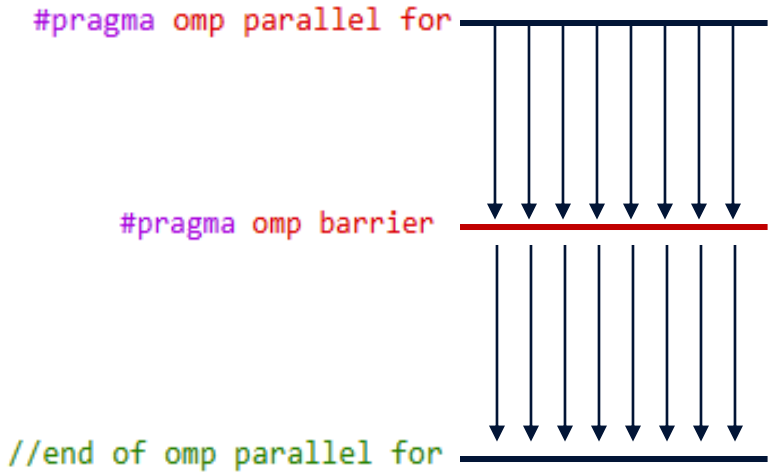
Synchronization and Barriers

- At the end of the region following a `#pragma omp parallel`, OpenMP automatically joins all threads
  - i.e. implicit barrier

- You could introduce **artificial barriers** with the following pragma

```
#pragma omp barrier
```

  - All threads have to enter this point of the program before continuing
  - It is therefore a *collective* operation



- Barriers are rarely used in OpenMP applications
  - But their existence and discussion remind us the MIMD architecture that is programmed
  - Threads run **asynchronously** – threads are not run in lock-step

- In contrast OpenMP's `#pragma omp simd` statement *is run in lock-step*
  - Can interpret it as a sequence of instructions where **each statement is followed by a barrier**
  - We will see its use later

LIBRARY FUNCTIONS

- Number of library functions – use for debugging
  - **`omp_get_max_threads`** - returns the number of threads in code. Should be same as `OMP_NUM_THREADS`
  - **`omp_get_num_threads`** - gives the number of threads that are actually used within a parallel environments. Can be less than the maximum number of threads.
  - **`omp_set_num_threads`** - set the number of threads manually. Can reduce the number of threads, but not able to exceed OMP_NUM_THREADS.
  - **`omp_get_thread_num`** - returns the current thread ID
- Avoid using OpenMP functions – these changes your original code with OpenMP specifics

OpenMP Philosophy
Stick as close to a serial code as possible and add all parallelization through `pragmas` only

FOR-LOOPS SUITABLE FOR OPENMP PARALLELIZATION

- Not every loop is well-suited for OpenMP parallelization
- Canonical form of OpenMP Loops :

  - **Countable** : loop trip-count known at the start of the loop (at runtime). Allows OpenMP to distribute a loop over multiple threads. Thus, while loops without a known count is ill-suited.
    No `#pragma omp while` for this very reason !

  - **Well-formed** : have a simplistic counter increment/decrement and the ranges may not change at runtime. No altering the loop counter within the loop. No breaks in loops/ jumps out of loop or throw exceptions

- However an OpenMP thread is a full logical von Neumann core
  - Complex branching and function calls in loop body allowed
  - A thread can arbitrarily jump within the loop body

- Related - **loop-carried dependencies** not a loop-form property – but a **program correctness property**
  - Programmer's responsibility to make sure no loop-carried dependencies before adding OpenMP pragma
  - Correctness is guaranteed by the programmer
  - [See later for more on loop-carried dependencies]

For-Loops suitable for OpenMP Parallelization

✔
```
#pragma omp parallel for
for (int i = 0; i < N; i++) {
    a[i] = 0.0;
}
```

✘
```
#pragma omp parallel for   // ERROR: cannot apply omp for to while
while (!done) {
    work();
}
```

✘
```
#pragma omp parallel for
for (int i = 0; i < n; i++) {
    if (a[i] < 0) n--;     // changes loop bound while running
}
```

✘
```
#pragma omp parallel for
for (int i = 0; i < N; i++) {
    if (a[i] == target) break;   // illegal for canonical omp for
}
```

✘
```
#pragma omp parallel for
for (int i = 0; i < N; i++) {
    if (bad(i)) throw std::runtime_error("oops"); // escapes parallel region
}
```

✔
```
#pragma omp parallel for
for (int i = 0; i < N; i += 1) {
    work(i);
}
```

✔
```
#pragma omp parallel for
for (int i = N - 1; i >= 0; i--) {
    work(i);
}
```

✘
```
#pragma omp parallel for
for (int i = 0; i < N; i = i * 2) {  // non-linear progression
    work(i);
}
```

✘
```
#pragma omp parallel for
for (int i = 0; i < N; i++) {
    if (a[i] == 0) i++;     // modifies i inside loop
}
```

✘
```
#pragma omp parallel for
for (int i = 0; i < N; i++) {
    if (a[i] < 0) goto done;     // jumps outside loop
}
done:
;
```

Thread Information Exchange

- Original BSP only allows information exchange at the **end of a super step**
  - Threads run completely independent of each other
  - Only at end they synch – and combine their results

- OpenMP allows for **threads to write to shared memory** during local computation (i.e. scratch pad)
  - Some threads can write to it, while others can read it
  - Leads to data races !

- Data races
  - **Read-write race:** one thread reads a shared variable before another thread has finished writing it.
  - **Write–read race:** one thread overwrites a shared variable before another thread has read the old value.
  - **Write–write race:** multiple threads write to the same shared variable, and the final value depends on which write happens last.

```
#pragma omp parallel for
for (int i = 0; i < 20; i++) {
    r += i; // read r, then write r : read-write race
}

#pragma omp parallel for
for (int i = 0; i < 20; i++) {
    r = i;
    x[i] = r; // expects "its own" r, but may get another
              // thread's value: Write-read race
}

int r;
#pragma omp parallel for
for (int i = 0; i < 20; i++) {
    r = i;  // write-write race on r
}
```

Critical Sections

- A **critical section** is a named sequence of instructions only one thread may execute at a time, forcing all other threads to wait until it finishes
  - A thread executing it "locks" the sequence
  - No other thread can process it while "locked"

```
#pragma omp parallel for
for (int i = 0; i < 10; i++) {
    #pragma omp critical
    {
        r += i;   // only one thread executes this at a time
    }
}
```

Serializes execution – still not good !

- Critical sections with the **same name share the same lock**, **different names use different locks**

```
#pragma omp critical (sectionA)
{ x *= 2; }

#pragma omp critical (sectionB)
{ y *= 2; }

#pragma omp critical (sectionA)
{ x /= 2; }
```

```
#pragma omp critical
{ x *= 2; }

#pragma omp critical
{ y *= 2; }
```

- Unnamed critical sections share **one global lock**

Critical Sections – Powerful but Dangerous !

- Critical sections are powerful - allows **fine-grained locking**
  - Different data → different locks
  - Enables complex logic: A and C must not overlap, but B may overlap with either

- But .. dangerous – can create **deadlocks**
  - Two threads can each hold one lock and wait forever for the other

- Use **debuggers** to find deadlocks
  - kill the execution once it hangs, and then study the back trace
  - Sporadic/non-deterministic deadlocks are however quite difficult to find !

- Critical sections are also **expensive**
  - Administering them causes **severe overhead**
  - Instead use **atomics** for protecting a single variable

```
#pragma omp critical(A)
{
    #pragma omp critical(B)
    {
        // ...
    }
}

#pragma omp critical(B)
{
    #pragma omp critical(A)
    {
        // ...
    }
}
```

ATOMICS

- Protects access to a **single individual variable**
  - Works only for **primitive data types** such as `integers, floats`.
  - Semantically not different to critical sections on a single statement – but **significantly faster**
  - **Natively** supported by hardware

```
#pragma omp atomic read
y = x;  // read from x protected

#pragma omp atomic write
y = x;  // write to y protected

#pragma omp atomic update
x++;    // both read write protected

#pragma omp atomic // default is update
x++;
```

- In an atomic update the RHS should not be a complex expression
  - The evaluation of the **`foo()-y`** is not thread-safe

```
#pragma omp atomic
x+=foo()-y
```

ATOMICS : CAPTURE

- Atomic update + returns variable content prior to the update
  - Accepts two statements: the backup of the value into another variable
  - Follow-up with modification

```
#pragma omp atomic capture
{
oldValue = x; x++;
}
```

- Can write a classic **Compare and Swap (CAS)** with captures
  - Atomic operation that updates a memory location ***only if*** **its current value matches an expected value**, and **returns the old value**
  - CAS - **fundamental primitive** for lock-free algorithms - prevents race conditions **without locks**

```
#pragma omp atomic capture
{
    oldValue = x;    // capture old value
    x = y;           // attempt update
}

if (oldValue == comparison) {
    // CAS succeeded
} else {
    // CAS failed
}
```

SCALAR PRODUCT WITH ATOMICS AND CRITICAL SECTIONS

- The following loop computing scalar product has data races :

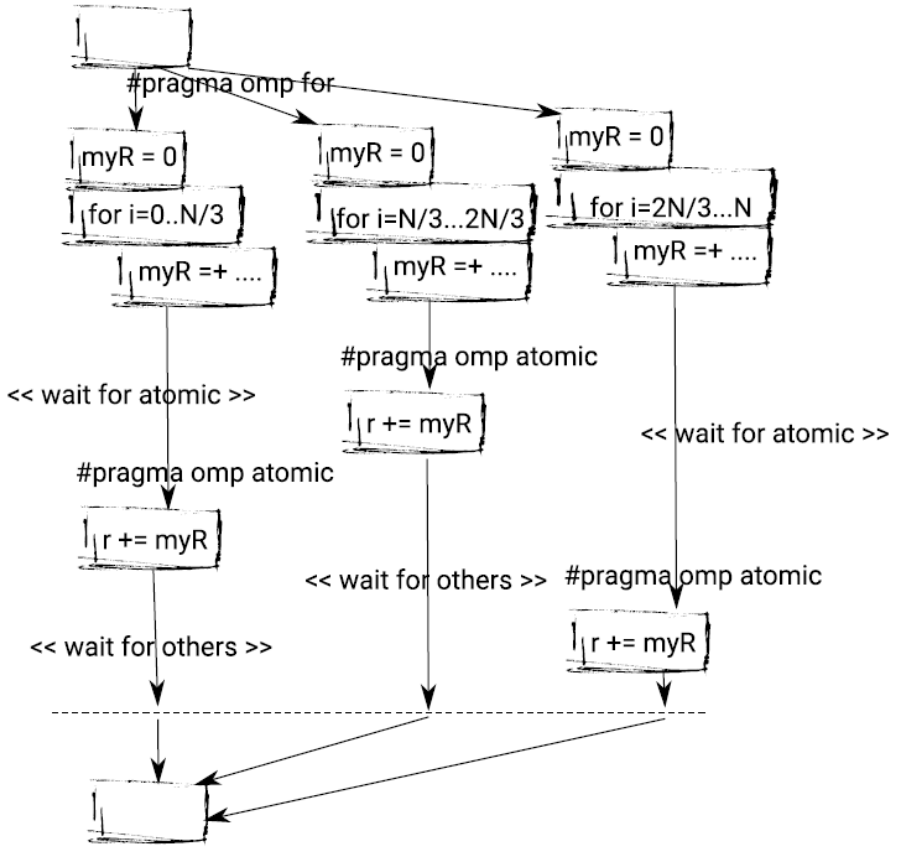
```
#pragma omp parallel for
for (int i=0; i<20; i++) {
  r += x[i] * y[i]; // can you spot the data races here ?
}
```

- Use atomics to protect :

```
#pragma omp parallel for
for (int i=0; i<20; i++) {
  double tmp = x[i] * y[i]; // some concurrancy
  #pragma omp atomic
  r += tmp; // serialized accumulation
}
```

- Improve further :

```
#pragma omp parallel
{
  double myR = 0.0;
  #pragma omp for
  for (int i=0; i<20; i++) {
    myR += x[i] * y[i]; // runs in parallel
  }
  #pragma omp atomic
  r += myR;
}
```



All-time classic of parallel computing
but breaks OpenMP Philosophy !

Variable Visibility

- All variables **declared outside** the `#pragma omp parallel for` are **shared**
  - loop counter and individual thread iteration range not shared

```
#pragma omp parallel for
for (int i = 0; i < N; i++) {
```

  - If variable outside parallel region then no need to tell OpenMP that its shared, though you can do so !

```
double r = 0;
#pragma omp parallel for shared(r)
for(int i=0; i<size; i++) {
  r += x[i] * y[i];
}
```

- *Private* variables – are owned by the respective thread. Not visible to other threads.
  - Create with declaring variable within parallel region – then each thread creates its own variable
  - Or declare as `private`

```
double r = 0;
#pragma omp parallel for private(r)
for(int i=0; i<size; i++) {
  r += x[i] * y[i];
}
```

  - Each thread creates it **own instance** of `r` and works on this. At the end of the loop all **local copies are destroyed and lost**. (however, this breaks the code semantics of above)

VARIABLE VISIBILITY

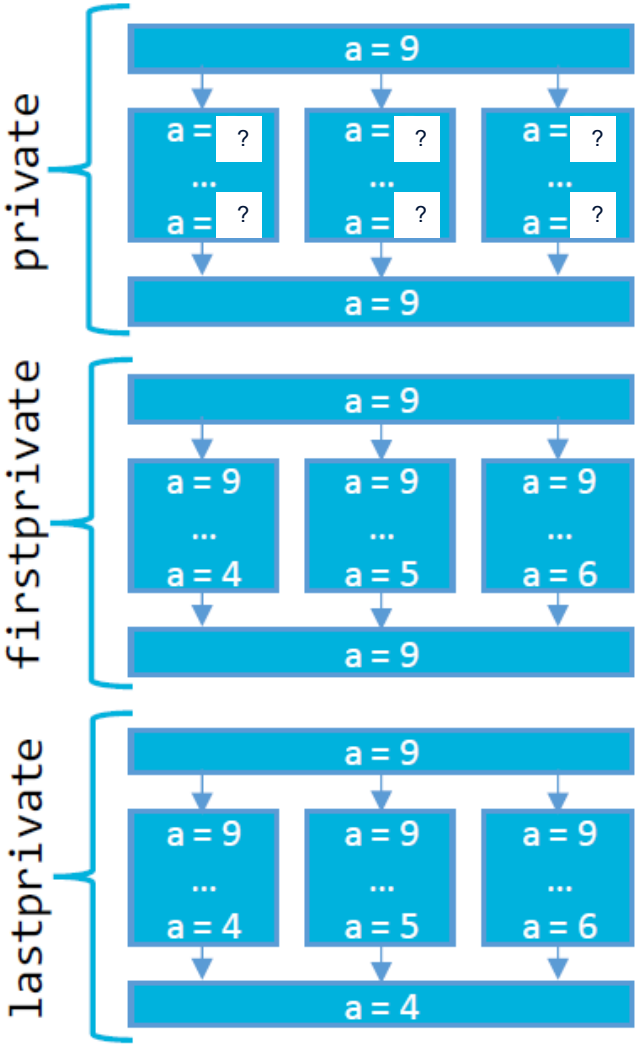
- Private variables - how are they initialized ?
  - No default initial values in C: thus it holds garbage !
  - Initialize ourselves within the loop body ?

- Use `firstprivate()` clause to initialize
  - Each local variable initialized with a copy of the original variable

```
double r = 0;
#pragma omp parallel for firstprivate(r)
for(int i=0; i<size; i++) {
  r += x[i] * y[i];
}
```

- There is also a `lastprivate()`
  - copies the value from the lexically **last iteration** executed in the original serial order to the variable after the parallel region

```
int x = 0;
#pragma omp parallel for lastprivate(x)
for (int i = 0; i < 5; i++) {
    x = i;
}
printf("%d\n", x);   // prints 4
```



DATA-SHARING RULES : default (none)

- `default(none)` **disables all implicit data-sharing rules** inside an OpenMP construct
  - Every variable used in the region **must be explicitly listed** in a data-sharing clause (`shared`, `private`, `firstprivate`, `reduction`, etc.).
  - If you miss one, the compiler emits an **error**.

```
#pragma omp parallel for default(none)
for (int i = 0; i < N; i++) {
    sum += a[i];   // error: a not scoped
}
```

```
double tmp;

#pragma omp parallel for default(none) shared(a,b,N) private(tmp)
for (int i = 0; i < N; i++) {
    tmp = b[i] * 2.0;
    a[i] = tmp;  // Without private(tmp), this would be a data race
}
```

REDUCTIONS

- Reductions allow collective communication
  - Synchronize + move information + compute

- OpenMP `reduce` clause if specified occur at the end of the BSP super-step

```
#pragma omp parallel for reduce(+:r)
for(int i=0; i<N; i++) {
  r += x[i] * y[i];
}
```

```
double r = x[0] * y[0];
#pragma omp parallel for reduction(max:r)
for (int i = 1; i < N; i++) {
    double v = x[i] * y[i];
    if (v > r) r = v;
}
```

```
double r = x[0] * y[0];
#pragma omp parallel for reduction(min:r)
for (int i = 1; i < N; i++) {
    double v = x[i] * y[i];
    if (v < r) r = v;
}
```

- variable(s) specified with an associative and commutative operator
  - Compiler declares variable as **private** – each thread has its own copy
  - Copy **initialized to zero** (for +) **or 1** (for *)
  - All accumulations are thread-local, only result is reduced into the original variable at the end
  - **Multiple variables** can be specified for reduction separated by a comma.

- **Significantly faster** than manual implementation of a reduction
  - Might reduce with a binary tree or other optimized method
  - However output is **non-deterministic** (we discussed this before)

Grain Sizes and Scheduling

- OpenMP's `parallel for` leaves it to the OpenMP runtime to decide how the iteration range is split
  - Looks at the `OMP_NUM_THREADS` environment variable and
  - Defaults to simplest heuristic to split loop : **static scheduling**
  - Scheduling – deciding which work is done by which thread

- **Grain size:** the number of loop iterations grouped together and assigned to one thread as a single work unit
  - smaller grain size → more parallelism but higher overhead
  - larger grain size → less overhead but potentially poorer load balance

Grain Sizes and Scheduling

- **Static scheduling :**
  - A work set with *N* items is cut into pieces (grains) of size *N/p (N > p)* or 1 (N ≤ p) respectively
  - pieces are then distributed among the *p* compute units
  - Distribution not changed at/during runtime
  - Static Scheduling helps to eliminate unreasonable small work units per thread

- Can constrain the grain size with the `schedule` clause

```
#pragma omp parallel for schedule(static,20)
```

  - Ensures that no chunk of the iteration range is bigger than 20
  - Can lead to over decomposition and ill-balance or underutilization of threads

Grain Sizes and Scheduling

- **Dynamic scheduling** :OpenMP threads are allowed to **steal work** from other threads if they idle.
- Loop iterations many have unequal runtimes – severe **load imbalances**
  - Static partitioning assign mostly expensive iterations to one thread and cheap ones to others
  - Solution: dynamic scheduling splits work into small chunks and assign then on demand

- Threads that finishes early **steal** remaining work from others
  - Trade-off : improves load balance but introduces **scheduling overhead**

```
#pragma omp parallel for schedule(dynamic,2)
```

  - E.g. split work into chunks of 2 and equally distribute among threads – as soon as a thread finishes its work, it can steal from others
  - Use Dynamic scheduling only if **per-loop iteration times fluctuate significantly**.

- **Guided scheduling** : dynamic but starts with large chunks and gradually gives out smaller chunks
  - Low overhead early (large chunks) + good load balance at the end (small chunks)

```
#pragma omp parallel for schedule(guided, 2)
for (int i = 0; i < N; i++) {
    work(i);   // varying cost per i
}
```

Grain size does not go below 2

TILING : `#pragma tile sizes(n,n)`

- **OpenMP loop-transformation directive** that **breaks a loop nest into tiles**
  - **Tiling** groups iterations into rectangular blocks so that data reused within a block stays in cache longer

```
for (int j = 0; j < M; j++)
  for (int i = 0; i < N; i++)
    A[j][i] += 1.0;
```

Manual →

```
for (int jj = 0; jj < M; jj += 32)
  for (int ii = 0; ii < N; ii += 32)
    for (int j = jj; j < min(jj+32,M); j++)
      for (int i = ii; i < min(ii+32,N); i++)
        A[j][i] += 1.0;
```

↓ Automatic via OpenMP

```
#pragma omp tile sizes(32,32)
for (int j = 0; j < M; j++)
  for (int i = 0; i < N; i++)
    A[j][i] += 1.0;
```

Parallel →

```
#pragma omp parallel for
#pragma omp tile sizes(32,32)
for (int j = 0; j < M; j++)
  for (int i = 0; i < N; i++)
    A[j][i] = B[j][i] + C[j][i];
```

outer tiles are distributed across threads
inner tile loops run sequentially per thread

- Requires OpenMP 5.0+
- It's a performance hint, no guarantee
- Tile size should be chosen so that the **data reused inside a tile fits into a fast cache level**, maximizing reuse and minimizing memory traffic.

Loop-carried Dependencies

```
for (int i = 1; i < n; i++) {
    a[i] = a[i] + c;          // statement 1
    b[i] = a[i - 1] * b[i]; // statement 2 (loop-carried RAW on a)
}
```

- No dependence between statement 1 and statement 2 **within a single iteration** of the loop.
- But there is a dependence between **two successive iterations:**
  - When $i = k$ the s_2 reads the value of *a[k-1]* written by s_1 in iteration *k-1*

- Adding `#pragma omp parallel for` will give incorrect results
  - Iterations cannot be re-ordered in this case.
  - OpenMP **trusts the programmer** to not have loop-carried dependencies.

More Loop-carried Dependencies

- Flow dependence (also called a **true dependence**)
  - Iteration *i* **reads** a value that iteration *i-1* (or earlier) **wrote** - i.e. Read after Write (RaW)

```
for (int i = 1; i < n; i++) {
    a[i] = a[i-1] + c;   // uses a[i-1] produced by previous iteration
}
```

Difficult to remove – need to change algorithm !

- Anti dependence
  - Iteration *i* **writes** a location that an earlier iteration *i-1* (or earlier) **read** - i.e. Write after Read (WaR)
  - Not a "true" dependence as its caused by reusing the same storage location

```
for (int i = 0; i < n-1; i++) {
    t = a[i];        // READ a[i]
    a[i] = b[i+1];   // WRITE a[i] (later iterations read a[i] too, etc.)
}
```

Often removable by variable renaming storage / privatization - each iteration gets its own

e.g. replace `t` with `t_arr[i]`

- Output dependence
  - Multiple iteration **write** the same location; final value depends on which write happens last - i.e. Write after write (WaW)

```
for (int i = 0; i < n; i++) {
    y = f(i);   // WRITE y every iteration
}
```

Fix by
- **making `y` private**
- **Use `reduction` if combining y**
- **Store into `y[i]` if all results needed**

DEMO – SOLUTION TO 2D LAPLACE EQUATION USING THE JACOBI METHOD

- 2-D steady Laplace equation with Dirichlet boundary conditions using the Jacobi iterative method

PDE: $$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0$$

BCs: $$\begin{cases} u(0,y) = \sin(\pi y) \\ u(1,y) = \sin(\pi y)e^{-\pi} \\ u(x,0) = 0 \\ u(x,1) = 0 \end{cases}$$

Discrete update: $$u_{i,j} = \frac{1}{4}(u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1})$$

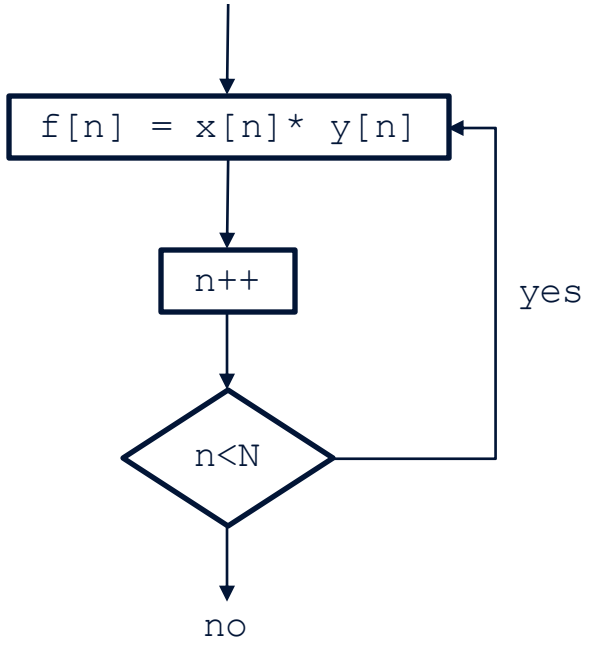
SIMD WITH OPENMP

- Recall SIMD / Vector computing as a **very special case of BSP**
  - where **each instruction** forms its own super-step
  - the join's synchronization happens **implicitly after the instruction**

- SIMD vectorization : <u>same instruction</u> operates on **multiple independent data elements** simultaneously

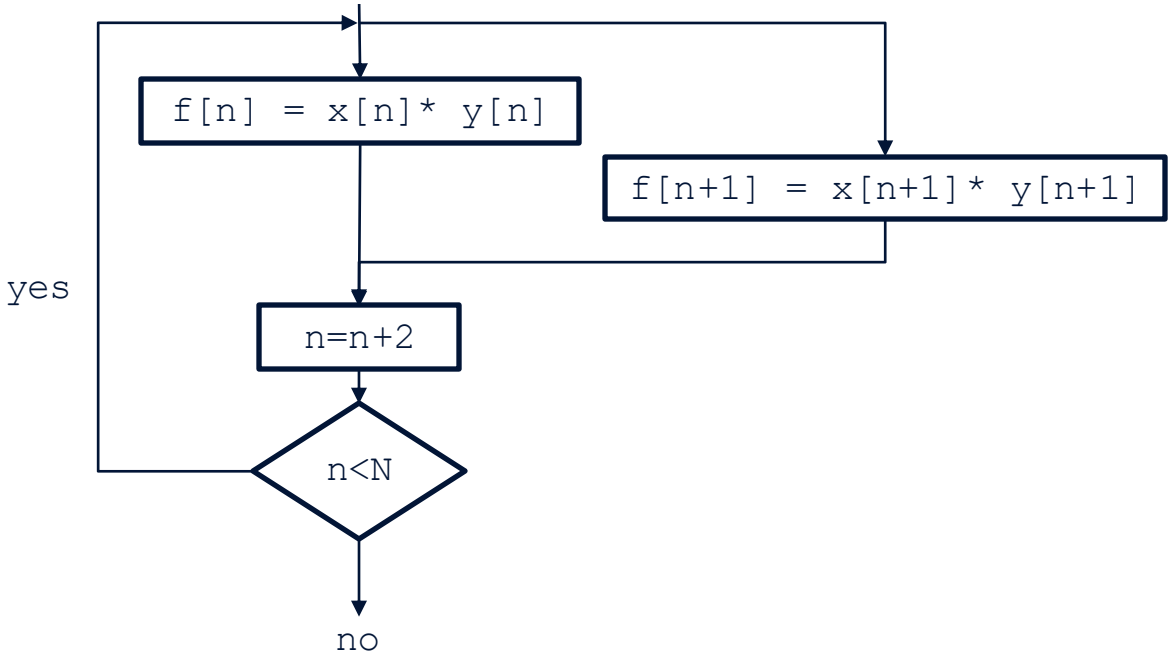
```
for (int n=0; n<N; n++) {
  f[n] = x[n] * y[n];
}
```

Loops look sequential even though iterations are independent




```
for (int n=0; n<N; n+=2) {
  f[n]   = x[n]   * y[n];
  f[n+1] = x[n+1] * y[n+1];
}
```

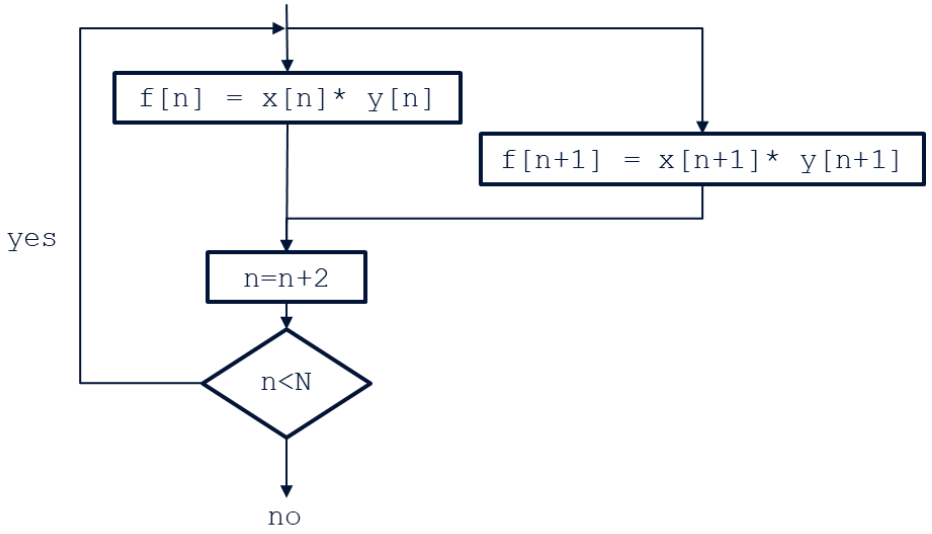
Manually unrolling shows the independent iterations

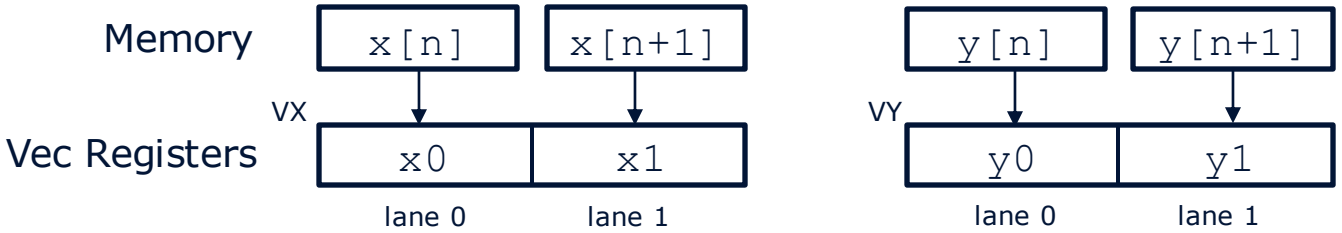


SIMD IN ACTION

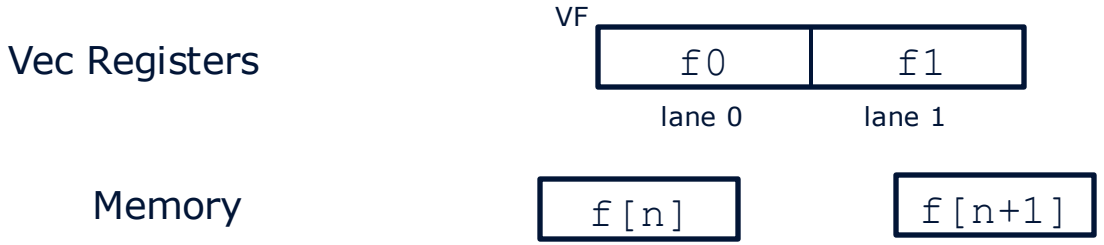
- For 2-lane SIMD:

```
for (int n=0; n<N; n+=2) {
  f[n]   = x[n]   * y[n];
  f[n+1] = x[n+1] * y[n+1];
}
```





Mul (vector) `VF = VX * VY = [x0*y0  x1*y1]`




```
VX <- LOAD2( &x[n] )        // VX = [x[n],   x[n+1]]
VY <- LOAD2( &y[n] )        // VY = [y[n],   y[n+1]]
VF <- VMUL( VX, VY )        // VF = [x[n]*y[n], x[n+1]*y[n+1]]
STORE2( &f[n], VF )         // f[n], f[n+1] written together
n  <- n + 2
```

Vectorization is not something the compiler invents — it is a property that (some) loops allows

SIMD WITH OPENMP

- Once you identify **SIMD-fiable loops** – can use OpenMP SIMD `pragma` to action the vectorization

```
#pragma simd
for (int n=0; n<N; n++) {
  f[n] = x[n] * y[n];
}
```

- No loop-carried dependencies
- contiguous memory
- Same operation each iteration

```
#pragma omp simd
for (int i = 0; i < N; i++) {
    y[i] = alpha * x[i] + y[i];
}
```

- Fused arithmetic – classic AXPY kernel
- Scalar `alpha` broadcast to all lanes
- Maps to a single FMA operation

```
#pragma omp simd
for (int i = 0; i < N; i++) {
    out[i] = a[i] + b[i] + c[i];
}
```

- Multiple arrays, same index
- All load/stores stride-1
- Assume no aliasing – use `restrict`

```
#pragma omp simd
for (int i = 0; i < N; i++) {
    x[i] += dt * vx[i];
    y[i] += dt * vy[i];
    z[i] += dt * vz[i];
}
```

- Struct of Arrays (SoA)
- Each field is a contiguous array
- Multiple independent vector loops

```
#pragma omp simd
for (int i = 0; i < N; i++) {
    y[i] = x[2*i];
}
```

- Strided access
- Requires gather loads
- Efficient only on newer CPUs

- You can explicitly limit SIMD vector length with `simdlen(length)` clause
  - Compiler chooses minimum of (simdlen, hardware width)
  - E.g. AVX512 supports 16 floats, `simdlen(8)` → only 8 lanes used
  - Gives **performance control without intrinsics**

```
#pragma omp simd simdlen(8)
for (int i = 0; i < N; i++) {
    y[i] = a[i] * b[i];
}
```

LOOPS SUITABLE FOR SIMD

- Canonical form of Loops for SIMD :
  - **Countable** : loop trip-count known at the start of the loop (at runtime). While loops without a known count will not vectorise.
  - **Well-formed** : have a simplistic counter increment/decrement and the ranges may not change at runtime. No altering the loop counter within the loop. No breaks in loops/ jumps out of loop or throw exceptions
  - **Branching/simplicity** : loop body has to be simple. **Conditionals** discouraged (though it might work with masking). **Inner loops only** – not both outer and inner. **Function calls** can be problematic. [see later on all of these issues]
- Additionally : If the compiler finds **loop-carried dependencies** then the `#pragma omp simd` will be rejected (more on this later)
  - Contrast this to `#pragma omp parallel for`

OpenMP SIMD pragma – Further clauses : `safelen(k)`

- We saw how SIMDisation can be read as **loop unrolling**
  - Then Vectorization ≈ unrolling loop by k and executing those k iteration in parallel

- How many iterations can be safely executed in parallel ?
  - Specify with **`safelen(k)`**

```
// a[i] depends on a[i-4]  (distance D = 4)
void recur_distance4(float *a, const float *b, int N)
{
    // Need a[0..3] initialized before the loop (boundary / seed values).

    #pragma omp simd safelen(4)
    for (int i = 4; i < N; i++) {
        a[i] = 0.5f * a[i - 4] + b[i];
    }
}
```

  - The compiler might refuse vectorization as `a[i]` and `a[i-4]` are loop-carried dependencies
  - Dependence distance = D, thus can have k ≤ D
  - `safelen(D)` means "no dependence within D"

OpenMP SIMD pragma – Further clauses : `uniform(var)` and `linear(var)`

- Indicate variables that will be identical across all vector lanes – `uniform(var)`

```
#pragma omp simd uniform(alpha)
for (int i = 0; i < N; i++) {
    y[i] = alpha * x[i];
}
```

  - `alpha` is identical for all lanes
  - Compute it once and broadcast it

- Inform compiler that memory access is **regular and predictable: linear(var)**
  - Generate clean vector loads
  - Avoid conservative assumptions

```
void copy(float *x, float *y, int N)
{
    float *px = x;

    #pragma omp simd linear(px:1)
    for (int i = 0; i < N; i++) {
        y[i] = *px;
        px++;
    }
}
```

Without `linear(px:1)`:
- compiler must assume: `px` could jump unpredictably
- vector loads might alias or overlap

With `linear(px:1)`:
- `px` advances exactly one element per iteration
- Vector loads are safe and contiguous

SIMD is **conservative by default**. OpenMP SIMD lets you inject domain knowledge and **remove compiler doubt** !

OpenMP SIMD pragma – Further clauses : `collapse(n)`

- **Nested loops** can lead to under utilization of SIMD lanes
  - Compiler will only vectorise the innermost loop

```
// I is small
#define I 3
#define J 1000

#pragma omp simd
for (int j = 0; j < J; j++) {
    for (int i = 0; i < I; i++) {
        y[j][i] = x[j][i] * alpha;
    }
}
```

- Assume inner loop length = 3
- SIMD width = 8
- Vectorization ineffective

```
// I is small
#define I 3
#define J 1000

#pragma omp simd collapse(2)
for (int j = 0; j < J; j++) {
    for (int i = 0; i < I; i++) {
        y[j][i] = x[j][i] * alpha;
    }
}
```

- Effective loop length = 3000
- SIMD fully utilized!

- If loops are **independent**, we can treat the 2D iteration space **as one long 1D loop**
  - Conceptually : `(i,j)` pairs → linear index `k`

```
#pragma omp simd collapse(2)
for (int j = 0; j < J; j++) {
    for (int i = 0; i < I; i++) {
        a[j][i] = b[j][i] + c[j][i];
    }
}
```

Conceptually →

```
for (int k = 0; k < J * I; k++) {
    j = k / I;
    i = k % I;
    a[j][i] = b[j][i] + c[j][i];
}
```

OpenMP SIMD pragma – Reductions

- Recall that a **reduction** computes **one value from many iterations**, e.g.:
  - Each iteration updates the same variable
  - Each iteration depends on the previous value of `y`
  - Naively, this **prevents vectorization**

```
double y = 0.0;
for (int n = 0; n < N; n++) {
    y += x[n];
}
```

- However reductions can be restructured
  - Multiple partial sums, followed by a final combination
  - The `#pragma omp simd reduction()` instructs the compiler to do this

```
#pragma omp simd reduction(+:y)
for (int n = 0; n < N; n++) {
    y += x[n];
}
```

Conceptually →

```
lane0: y0 += x[0], x[4], x[8], ...
lane1: y1 += x[1], x[5], x[9], ...
lane2: y2 += x[2], x[6], x[10], ...
lane3: y3 += x[3], x[7], x[11], ...

y = y0 + y1 + y2 + y3
```

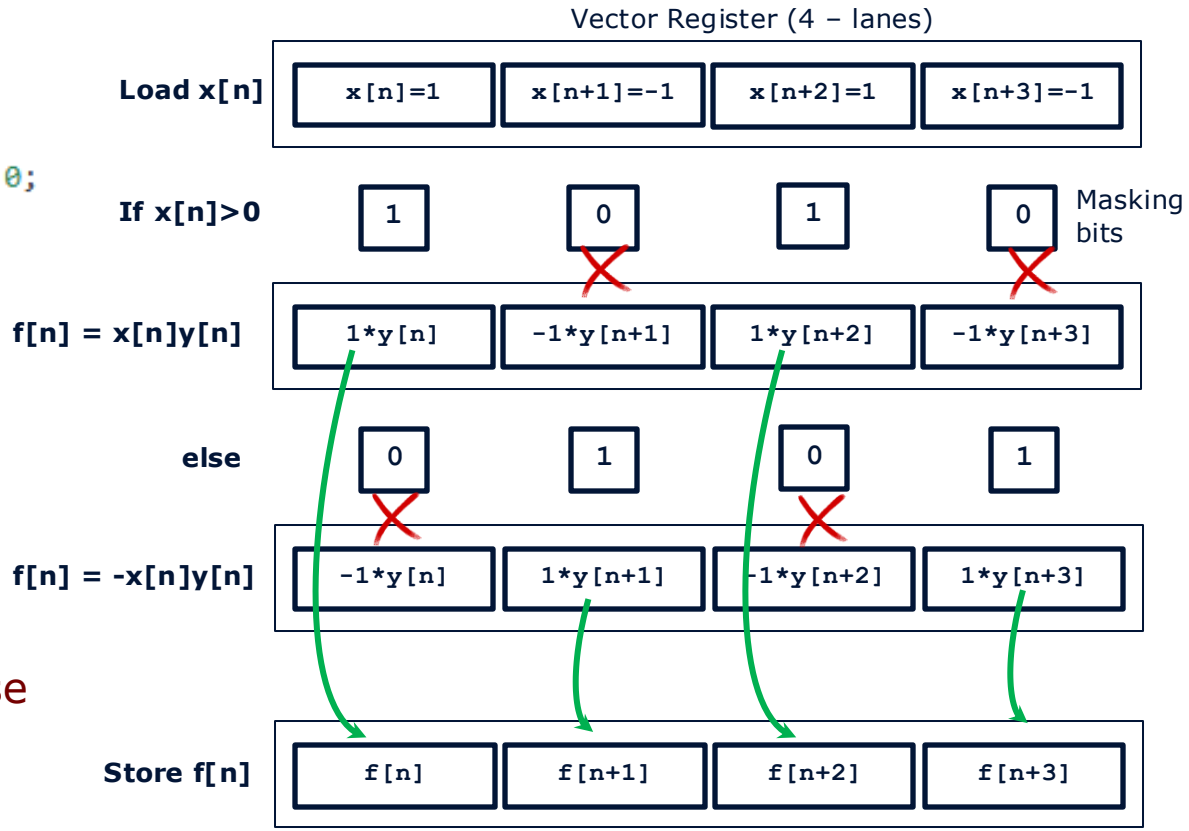
  - Compiler generates - blocked scalar product plus horizontal SIMD
  - Floating-point addition is **not strictly associative** - the result may differ slightly from scalar execution
  - Supports operations - + (sum), * (product), -(minus), logical and bit-wise and (& and &&), or (| and ||) and xor (^), min and max
  - Compiler generates - blocked scalar product plus horizontal SIMD
  - You can also **reduce several parameters** in one rush; enlist them separated by a comma.

- SIMD vectorization can be applied to **simple branching code** – this is achieved through **masking**
  - The conditional introduces a **divergence** – handled via masking/blending

```
#pragma omp simd
for (int n = 0; n < N; n++) {
    if (x[n] > 0.0) {
        f[n] =  x[n] * y[n];
    } else {
        f[n] = -x[n] * y[n];
    }
}
```

```
#pragma omp simd
for (int n = 0; n < N; n++) {
    double s = (x[n] > 0.0) ? 1.0 : -1.0;
    f[n] = s * x[n] * y[n];
}
```

- Often faster : avoids the explicit branch and usually vectorizes more reliably

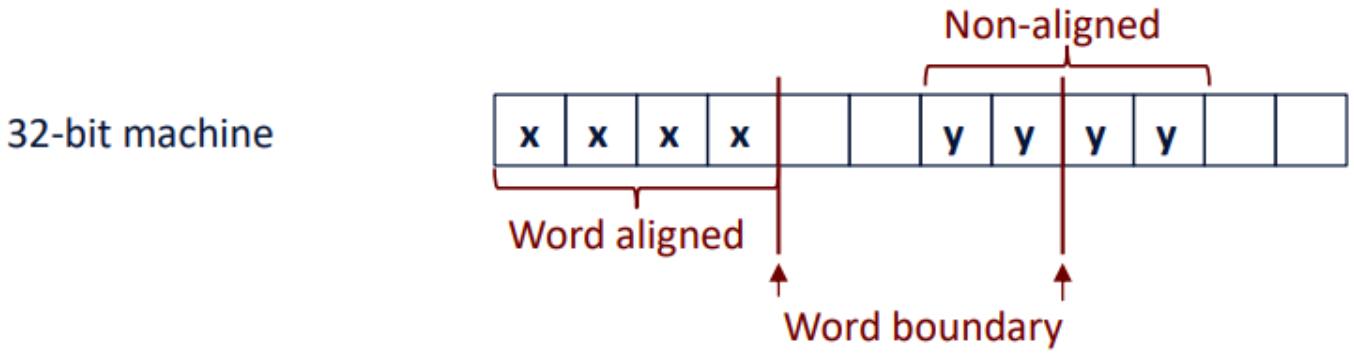


- Masking – condition is evaluated once using its result to enable or disable individual SIMD lanes.
  - Each lane **computes in parallel**, but only lanes whose mask is **true** keep their result; the others are ignored.

- Blending – selects between **two already-computed values** using a mask
  - Masking controls *whether* a lane participates, blending controls *what value* a lane receives

CONTIGUOUS DATA AND ALIGNMENT

- SIMD is fast when data is **contiguous** and **aligned**



- SIMD can only load **consecutive values** efficiently : `x0 x1 x2 x3 x4 x5 x6 x7` ← ideal (1x vector load)
  - If not consecutive need **multiple loads** or use special (slower) **gather** instructions
  - **Gathers** load values from different memory locations and pack them into one vector register
  - Conversely **scatters** write vector lanes back to different memory locations

- **Unaligned access** may require multiple loads – historically **more expensive**
  - Use `memalign()` and `posix_memalign()` for C to allocate aligned memory

```
#pragma omp simd aligned(x,y:64)
for (int i = 0; i < N; i++) {
    y[i] = x[i];
}
```

Once aligned memory is guaranteed, tell the compiler.

Contiguous DATA and Alignment

❑ SoA is SIMD friendly

```
void step_soa(int N, float dt,
              float *restrict x, float *restrict y,
              float *restrict z,
              const float *restrict vx,
              const float *restrict vy,
              const float *restrict vz)
{
    #pragma omp simd
    for (int i = 0; i < N; i++) {
        x[i] += dt * vx[i];
        y[i] += dt * vy[i];
        z[i] += dt * vz[i];
    }
}
```

- `x[i]` and `y[i]` are contiguous
- Can do aligned vector loads/stores

❑ AoS needs gathers

```
typedef struct {
    float x, y, z;
    float vx, vy, vz;
} Particle;

void step_aos(int N, float dt, Particle *p)
{
    #pragma omp simd
    for (int i = 0; i < N; i++) {
        p[i].x += dt * p[i].vx;
        p[i].y += dt * p[i].vy;
        p[i].z += dt * p[i].vz;
    }
}
```

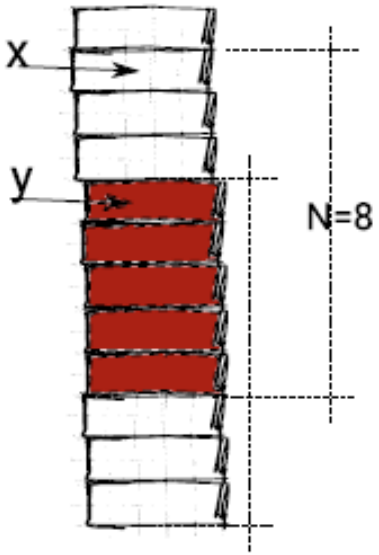
- Needs to gather x values from non-contiguous memory addresses

Issues Blocking SIMD Vectorization - Aliasing

- Aliasing - two memory locations can be accessed via different variables (pointers)

```
void foo(int* x, int* y, int newN) {
  for (int n=0; n<newN; n++) {
    x[n] = y[n];
  }
}
```

Compiler cannot be certain that `x` and `y` are not pointing to overlapping memory locations




```
void foo(int* restrict x, int* restrict y, int newN) {
  for (int n=0; n<newN; n++) {
    x[n] = y[n];
  }
}
```

Programmer should give assurance that there is no aliasing with the `restrict` key word

```
void foo(int* restrict x, int* restrict y, int newN) {
  #pragma omp simd
  for (int n = 0; n < newN; n++) {
    x[n] = y[n];
  }
}
```

Now can use `omp simd` pragma to vectorize

Compiler feedback will inform if aliasing is preventing vectorization (see flags later)

- Also can use `safelen` clause to inform distance that's safe to vectorize

Issues Blocking SIMD Vectorization - Function calls within a SIMD loop

- Function calls within a loop prevent a compiler to vectorise the loop
  - Use **inlined functions** – replace function call with function's body, parameter passing becomes assignments
  - Inform OpenMP that a function is going to be used within a loop – `#pragma omp declare simd`
  - Ensure that SIMDised function is **reasonably simple** – help compiler create vector variant

```
// properly prepared min function
#pragma omp declare simd
  int min (int x, int y) {
    return x < y ? x : y;
  }

// take element-wise min of entries from a/b and store in y
void min(int* x, int* y, int* z, int N) {
#pragma omp simd
  for (int n=0; n<N; n++)
    z[n] = min(x[n], y[n]);
}
```

Issues Blocking SIMD Vectorization - Loop carried Dependencies

- Recall loop-carried dependencies

```
for (int i = 1; i < n; i++) {
    a[i] = a[i] + c;          // statement 1
    b[i] = a[i - 1] * b[i]; // statement 2 (loop-carried RAW on a)
}
```

  - For `omp simd`, loop-carried dependencies are a **compile-time legality issue**
  - #pragma omp simd will **reject SIMD** if compiler cannot prove no dependencies

- In contrast `#pragma parallel for` will parallelize but will give **incorrect results**
  - It trusts the programmer to do the right thing !

SIMD enforces dependency rules more strictly !

OpenMP Parallel for vs SIMD

	`#pragma omp parallel for`	`#pragma omp simd`
Parallel Unit	CPU cores	Vector lanes
Parallelism Level	thread-level parallelism across cores	vector parallelism inside a core
Start-up cost	High (thread creation/scheduling)	Low
Control flow	Independent threads	Lockstep execution
Branch divergence	OK	Bad (wastes lanes)
Loop body cost/complexity	Can be expensive / complex	Must be cheap / simple and uniform
Best use	Outer Loops	Innermost loop
Loop-carried dependancies	Incorrect results	Rejects SIMD

- Both need *enough work* to amortize overhead
- Can be nested – `simd` should be within `parallel for`
- Divergence is an issue with SIMD
- Ill-balanced threads is an issue in parallel for

```
#pragma omp parallel for
for (int j = 0; j < N; j++) {
  #pragma omp simd
  for (int i = 0; i < M; i++) {
    a[j][i] += b[j][i] * c[j][i];
  }
}
```

Node Architecture Terms and Jargon !

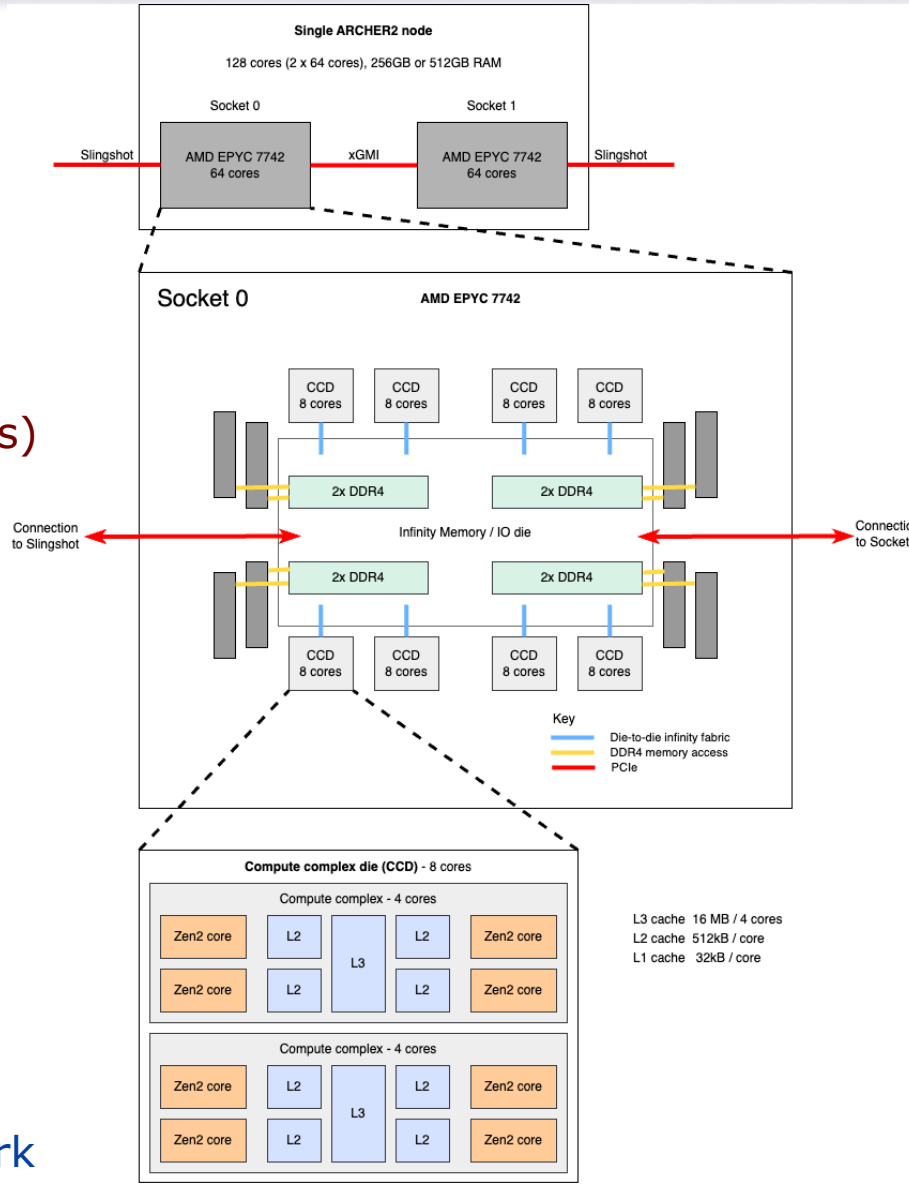
- Compute ***core*** – CU + ALU + FPU + Cache + [shared bus and memory]
- *software* (logical) ***thread*** *vs hardware* ***thread*** (i.e. core)

- Multiple **logical threads** running on a **core**
  - On a CPU swap context – not enough registers (done by the OS)
  - GPUs have way more logical threads than physical (hardware / cores) threads as it has significant number of registers

- *simultaneous multithreading (SMT)* – hardware cores share resources
  - Intel - *Hyperthreading*
  - Shared FPU ? For HPC/Scientific simulations its important to know

- *socket* - physical processor package (a piece of silicon) that contains **multiple cores and their directly attached memory**
  - Accessing memory of other sockets is non-uniform – **NUMA**

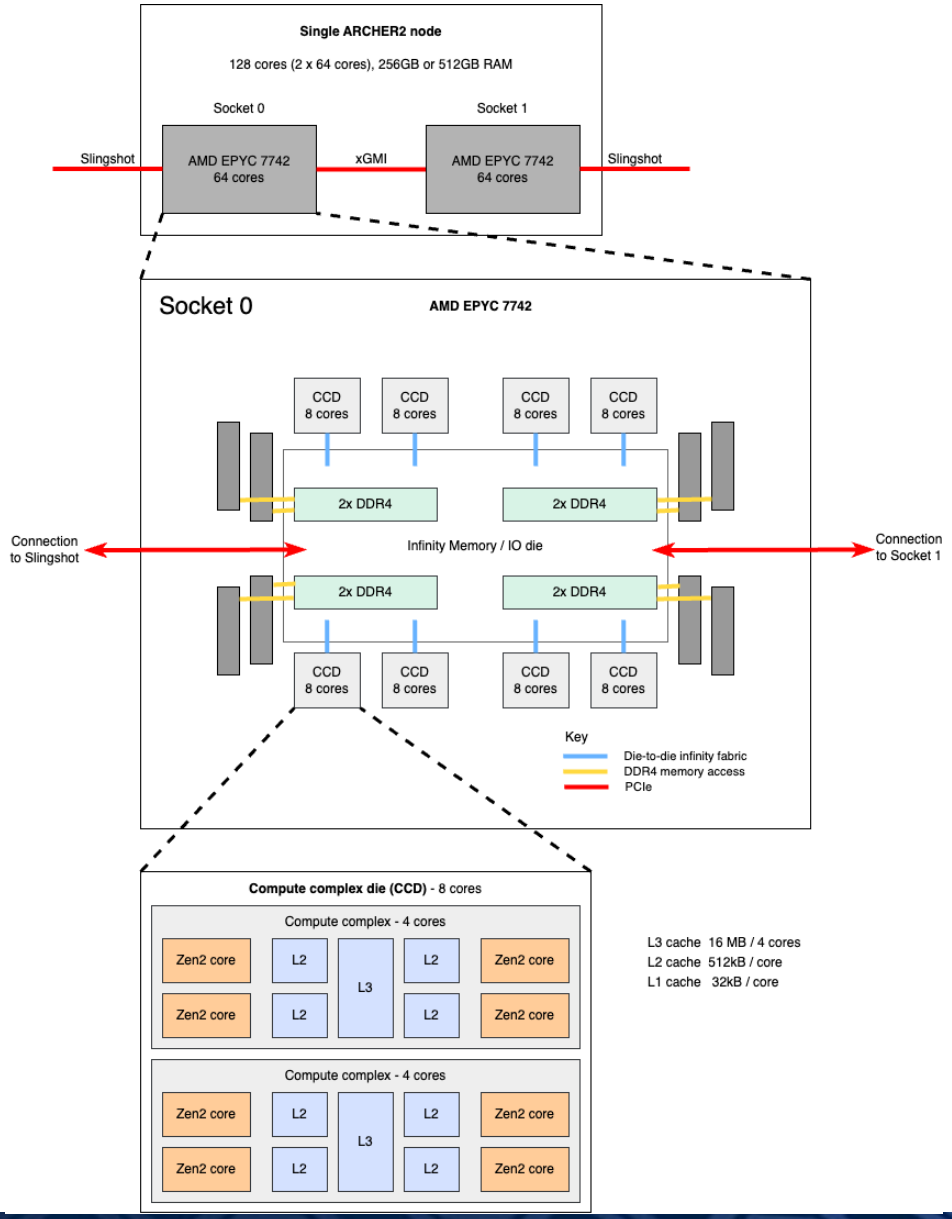
- *Node* - Consists of multiple sockets + RAM + Network
- *GPU Node* – Consists of Multiple GPUs + CPU sockets + RAM + Network



ARCHER2 – Node Architecture

Component	Details
Processor	2x AMD Zen2 (Rome) EPYC 7742, 64-core, 2.25 Ghz
Cores per node	128
NUMA structure	8 NUMA regions per node (16 cores per NUMA region)
Memory per node	256 GB (standard), 512 GB (high memory)
Memory per core	2 GB (standard), 4 GB (high memory)
L1 cache	32 kB/core
L2 cache	512 kB/core
L3 cache	16 MB/4-cores
Vector support	AVX2
Network connection	2x 100 Gb/s injection ports per node

https://docs.archer2.ac.uk/user-guide/hardware/



Gaining best Performance from OpenMP

- Check compiler flags :

`gcc: -O3 -march=native -fopenmp -fopt-info-vec-optimized`	`icx -O3 -xHost -qopenmp -qopt-report-phase=vec,par`

- Check Vectorization / Parallelization report from compilers : e.g. `-fopt-info-vec-optimized`
- Help compiler prove validity of paralleization / Vectorization :
  - removing aliasing
  - indicate alignment
  - make function calls SIMD-ready with declare simd
  - control SIMD width, `safelen()`
  - Eliminate **loop-carried dependencies** (if possible)
  - rewrite whole loop if need be !
- Make sure there is enough work for a `parallel for` – amortize overhead
  - minimize number of parallel regions
  - Use one outer `parallel` region and multiple `parallel for`
  - Avoid barriers use `nowait` (check OpenMP specification)

- Do not use shared data within a `parallel for` unless needed
- Use atomics instead of critical sections
- Play with the schedule clause

Gaining best Performance from OpenMP

- Thread pinning
  - `OMP_PROC_BIND=[spread, close]` – sets the binding of threads to processors
  - `OMP_PLACES=[cores, sockets, clustom-list]` – defines a list of places that threads can be pinned on
  - `OMP_DISPLAY_ENV=verbose` – check what runtime envs are
  - `numactrl`

spread threads as evenly as possible across the available places

bind only to physical core

```
e.g. : export OMP_NUM_THREADS=12;export OMP_PROC_BIND=spread; export OMP_PLACES=cores;
numactl --cpunodebind=0
```

- **False sharing –** threads write different data in the same cache line, causing unnecessary cache contention
  - see: https://www.openmp.org/wp-content/uploads/Booth_Klemm_Terboven.pdf
  - avoid multiple threads updating adjacent cache lines.

- Memory locality (**NUMA**)
  - First-touch initialize arrays in parallel (same thread that will use them).
  - Pair NUMA strategy with OMP_PLACES (sockets vs cores) depending on bandwidth vs locality.

DEMO WITH FURTHER PERFORMANCE

- 2D parabolic PDE solver from - https://people.maths.ox.ac.uk/gilesm/OpenMP/

$$\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}$$

- Solve on a unit square with homogeneous boundary conditions.
- Use regular grid with spacing of *h* in each direction, timestep of size *k*

- 5-point stencil computation:

$$U_{i,j}^{n+1} = U_{i,j}^{n} + \frac{1}{4}\left(U_{i+1,j}^{n} + U_{i-1,j}^{n} + U_{i,j+1}^{n} + U_{i,j-1}^{n} - 4\,U_{i,j}^{n}\right)$$

Further Reading and Material

- Tobias Weinzierl. *Principles of Parallel Scientific Computing, A First Guide to Numerical Concepts and Programming Methods*, Springer (2021). https://link.springer.com/book/10.1007/978-3-030-76194-3 **[Chapters 7, 9, 16]**

- Mike Giles - OpenMP Tutorial : https://people.maths.ox.ac.uk/gilesm/OpenMP/

- OpenMP Tasking - Tobias Weinzierl. *Principles of Parallel Scientific Computing, A First Guide to Numerical Concepts and Programming Methods*, Springer (2021). https://link.springer.com/book/10.1007/978-3-030-76194-3 **[Chapter 17]**